Video Transcripts

Module 21: Introduction to Reinforcement Learning and Deep Neural Networks

Video 1: Introduction (04:51)

So, this week we're going to continue our quest to understand machine learning and we're going to cover two techniques, reinforcement learning and also, deep neural nets. We're going to go into some depth with reinforcement learning, partly because it's one of my favorite methods and it was used by the AlphaGo team in Google to beat the world champion at Go and also the world champion at chess.

So, we're going to apply it to a robot that needs to find its way around. Now, in any of these situations, we need to build a map of where we are. So, even in, say, Go, we need a map of the state of the board. Now, in our case, we're going to have a robot that's finding its way through rooms, and we can only go if we're in room one, we can only go to certain other rooms, maybe we can go to two or to three, but we can't go to five or to six. So, one of the first things we need to understand is how we build up a map of our environment.

And the next thing is, we need to have a strategy for exploring. So, in the case of rooms, we have a number of, what we call, states. So, a state would be what room we're in. And then given that we're in a certain room, there are only certain actions that we can take. So, as I said, if we're in room one, we may be able to go to room two, but we can't go to room five. So, the first thing is we need to lay out the environment and have a way of expressing it.

So, we're going to express the rooms as a matrix. And, given that we're in a state, so we've got a matrix of states. And then at every state, we have a certain number of actions that are possible. So, if we're in room one, we can move to room two, but we can't move to rooms four, or five, or six, etc. Now, the robot needs to find its way to a goal. So, we need some way of expressing a goal and we need to explore. And so, we need to understand how we leave a trail, so that when we're successful, we need to somehow express that success.

© MIT xPRO® 2024. All rights reserved.

So, we're going to have what we call a robot matrix. And if we get to our goal, we're going to have a big robot. Now, we need to understand the quality of the state that we're in at any point. So, if we're close to our goal, we'll have a high quality, we call it a Q matrix. So, the value will be higher than if we're a long way from the goal, where we should have a lower quality. So, we're going to go through all of the techniques that machine learning for reinforcement learning uses. And once we've understood that, we're going to apply it to what's called the Frozen Lake problem. So, this is where a robot needs to find its way across a lake, but there are holes in the lake where the ice is melted, and if it goes there, it's going to fall in. So, we'll have fun with the Frozen Lake problem.

And then we're going to move to deep learning and we're going to use Colab. And, Colab is Google's version of Jupyter Notebooks, and we're going to use that to operate in the cloud. We're going to be using machines in the cloud via Colab to solve our problem. And in the deep neural net case, we're going to be doing a problem called Deep Dream, where we input an image of our self and we look at what the various layers in a deep neural net are doing to that image. So, we'll have kind of fun looking at how we're seen by the deep neural net. So, let's go and take a look at reinforcement learning and deep neural Nets. Bye for now.

Video 2: Understanding K-Means (07:49)

So, let's take a look now at an algorithm called K-Means that allows us to cluster blobs of data together. So, this is built into sklearn, but we're going to write our own algorithm, so that we can understand how it works. Now the first thing is that we can use a generator to make blobs of data. So here, we're going to make a number of clusters. Here, I'm making two clusters of data. So, we can change this number and make three clusters, four clusters, etc. Let me just show you. So, let's suppose we wanted four clusters. We would have four clusters. Let's go back to two clusters, and you can work through it with a different number. So, here's our two clusters of data. We can plot them out. So, this is saying we'll plot all the X's, and that's the first value, and this is the second value. So, if you like, it's an X, Y. So, we're just plotting X against Y here, and we've got 300 points.

So, let's take a look at how the algorithm works. So, now we call it and we say, okay, I want to divide this. I know I generated only two clusters, but I want to generate it into four. So, K-Means, we can just choose, and we have to choose how many clusters. It doesn't tell us

© MIT xPRO® 2024. All rights reserved.

automatically. Let's leave it at four here for now. So, it's dividing it into four clusters, and we can plot out the centers of those clusters. It turns out that the algorithm requires us to calculate the center, and what we mean by the center is we give each point the same weight and calculate where the centroid is of that cluster. So, we start off with four arbitrary points, and we cluster around those points and those points actually move. Let's take a look at an algorithm that we've written ourselves. So, this would be if we didn't want to use the sklearn one, we could write our own. So, what we're going to do is we're passing in the X, Y coordinates of all the points, and we say, how many clusters we're going to have. Now, we generate from all of the X readings. We generate a number of clusters. So, here we're going to call the algorithm eventually with a number of clusters. So, here find_clusters and we're passing in four. This rseed is just a seed that we use here. But when we see this, what we're going to get here for this i, let's print it out, we'll see that we get four points. So, we've got a whole string of 300 points, and we're going to choose the center where we're going to start for each cluster as whatever the coordinates of that 84th point are, the 217th, the 211th, and the 286th. So, these are arbitrary starting points for our centers, and in this case, for four clusters. We're going to keep the history of where those centers are because those centers are going to move. So, first we find the distance between all the points and the centers, the pairwise distances.

So, let me show you now, how we go about using the pairwise distance comparison to update the centers. Here, I've got two centers chosen at random, the blue one and the green one. Now, we calculate the pairwise distance of all the points, and you can see here that it's pretty obvious some of the points belong to the blue and some to the green, and a couple of them are quite close. So, what we do now is label the points according to which center they're closest to. So, here we've got the green ones and here the blue ones. Now, we update the center again. So, you can see the green centroid is not in the right place at the moment, it needs to be updated. So, we update it, and now what we see is that another point, this point here has become closer to the green center than to the blue center. So, this one will get updated and will be re-labeled as green and then again, the centroids will be moved. At some stage, no points are relabeled, and so everything is static and that's the end of the labelling algorithm. And so, the clusters then are well-defined, okay? So, we do this and then we calculate the centroid. So, what we're going to do here is we're going to calculate where the new centers are. Once, we've assigned the labels, we're going to calculate now the centroid of each of those labels. So, that'll be, and it'll move. And we're going to record now where it's moved to, in this history,

© MIT xPRO® 2024. All rights reserved.

and we go around again now and calculate the new distances of each point to those new centers. So, let's do it, and we'll see that we get a plot here, and what you can see is you can see the centroids moving. This green center started here, and then moved to the right and then came back a little. The yellow one looks like it started here, and moved a little to the left. You can see up here, this one moved from the left here towards the right for that cluster. And then, this one moved a little in that cluster. We stop our loop when we get no change of labels on points, when all the points stay statically attached to a given label, in this case a given color. It's a very simple algorithm, but quite powerful. So, why don't you play with this? So, for example, we can change the number of centers that we're going to find. So, let's see what would happen if we chose two centers. So, here we'd break up and you can see the centroid moving. So, in this case, it actually probably moved from the left here. It looks like this would be the centroid of these points. Okay, now suppose some of the points had color already assigned to them. How would that change our algorithm? So, now we've got points that are close, but they may be a different color. Well, it depends, because it's a little bit like comparing apples and oranges. In one case, we've got a distance, and in another case, we've got a color. So, unless we have some means of saying how we trade-off those metrics, it's difficult for us to choose how to find that distance.

Video 3: Using K-means to Classify Digits (05:09)

Now, we're going to apply K-Means to recognizing digits. So, there's a dataset of digits from zero through nine, and they're eight pixels by eight pixels. So, there's 64 pixels in each image, and each pixel has a grayscale number between and including 15. So, there's 16 Shades of grey. And we're going to load up this data and try and find out which images are similar to other images or which images are different. So, we need to have a metric of distance, D, between two images, A and B. So, say images eight and nine, we'd need to understand what's the difference. So, we're going to have to look at the grayscale color and take that into account. Now sklearns K-Means algorithm does a pretty good job of making a choice about that, but it's, as I say, it's not obvious how you trade off space with color. So, we load up the digits, the images, and we find there are 1797 of these images. Each image has 64 pixels. Each of those pixels has zero to 15 grayscale. So, each pixel, in some sense you can imagine, each pixel having a depth to it. Sometimes we talk about the the depth of the color, and here it's four bits, it's zero to 16.

And now, we'll apply the K-Means algorithm. We're going to choose 10 clusters because we know there are 10 digits, zero through nine. We ask K-means to cluster them, and the result is

© MIT xPRO® 2024. All rights reserved.

that we have 10 clusters. And so, let's take the center of each of those clusters. And so, that's a point in that pixel space if you like. And so, it'll look like an eight by eight image. So, here we plot out those centers. You can see most of the images there, one, two is a little, little shaky on two, and seven maybe a one, and eight here on the right is, that could be confused with the one. But you can see it does a surprisingly good job with this K-Means cluster. And in fact, here we're labeling each of these clusters. Now, it's arbitrary. Obviously, the cluster doesn't know what image it's looking at, it's just comparing grayscale and saying these are all close to each other.

Once we assign a label to a cluster, you can see that given any other image, we have about an 80% chance of identifying it, 0.795, so that's the accuracy score of given an image. We can label it and we'll be right 80% of the time. And we can also look at the confusion matrix. Now, we may not have come across this yet, so we've got 10 clusters, so zero through nine, and we're plotting which ones get confused. And you can see here, eight and one get confused. It has a score of 99. Five and nine get confused 42 times, eight and nine, 48. So, you see that there is some confusion and you see other ones here down at seven, quite low. But most of the time there is no confusion that we're identifying the image correctly. So, that's applying K-Means and you can see that it's quite a powerful algorithm and did a very good job with identifying these images. So, that's how you can apply it. So, K-Means clustering.

Why don't you run through these algorithms and make sure you understand them? There's some very neat Python structures in there, so take a look at those. In particular, I'm asking you to take a look at how this works. So, I want you to look up what this does, this permutation on random state, understand and take a look at what that i is. What's it doing? So, this is in find_clusters. I'll give you all this code and I hope you understand K-Means, it's quite a simple algorithm, but quite powerful. Okay, bye for now.

Video 4: Reinforcement Learning: States, Actions, and Rewards (08:48)

We're going to spend some time on reinforcement learning. It's a very powerful technique. In this case, we're going to look at a robot that needs to find its way around some space. So, here we've got a number of rooms labeled zero, one, two, three, four, and then the outside is five. And notice there are two doors to the outside. Now, we're going to need to navigate this and have the robot understand what its goals are so that when it starts in a certain place that it can find its goal. So, here we've got the start at room two, and so it would have to go through room

© MIT xPRO® 2024. All rights reserved.

three and one to go to finish that way, or it could go through three and four and get to finish this way. This outside is the same, it's one contiguous space. We'll come back to this. Let's discuss something a little bit simpler to begin with. So, suppose these are the rooms. Then, we'd call these the states. So, here might be our starting state, the yellow, and our goal might be to get to the green room here.

So, we could walk along here and up, or we could go up and along, but there may be some barriers in our way. So, we're going to need to think about a couple of things. We're going to need to think about how we represent geometry and also how we move and how we know what direction to move in. So, moving from one state to another is called an action. So, here's our starting state. And so, one action would be to move from this room to this middle room here and then to the end room and then up. But at any state, we need to know what's the best action to take. So, we could, for example, leave a trail. So, in this case, we'll label where we start and we'll explore and when we find a route to the goal that we'll label those steps, perhaps with a one. So, now the others might be labeled zero, these would all be labeled one. But if we then had to start in a different place, we'd have trouble because we wouldn't know whether to go up or to the right.

So, when we leave a trail, we need to have a value of a state. So, here we're in this state labeled three. And we know that if we go to two, that's not the best move, that we should go to the state with the value four. So, we've got these concepts of states and actions, and we've introduced maybe the value of a state. So, the question is, how would we assign values? Well, we might have a reward here for the goal. We might give it 100, the value of 100, and therefore now the other states would be labeled with lesser values, so that we'd know we should go towards that 100. Now, here's a more complex case where some of the rooms are walled off and you can't move through these walls. So, we somehow need to represent what actions are possible at any state. So, for example, if I'm in room three here, I can't move to room zero, I'd have to move to room . If I was at seven, I could move to room six to room eight or four. So, at any state, I've got a number of actions and we need to represent that somehow. First thing is we need to label the rooms, label the states.

So, zero, one, two, three, four, five, six, seven, eight. The way we label them doesn't really matter. We just need to know that they're labeled and be consistent thereafter. As I say, now we need to know what actions are possible given some state. Let's think about how we can

© MIT xPRO® 2024. All rights reserved.

represent what actions are possible at each room, at each state. Let's take state zero. We're in state zero. Now the question is, what are the possible actions? Well, let's suppose we don't know anything about the geometry. We know that possible actions are these. We could move to any of these rooms. Now, there's only some that are possible to move to, and in this case we can only move to one. But this is the way that we can represent a state and the places that it can move to; the actions. Now what about state one? Well, that would look like this. We're in state one. We could move to zero, we could move to two, or we could move to four, but we can't move to any of the others. So, we can put zero's in the places that we can't go and one's in the places that we can go.

And that will at least be a starting point that we know at the beginning. Just from looking at the state, we know that we can list the actions. So, typically we're going to have, in this case we've got nine states and there are nine possible actions. So, we're going to end up with a nine-by-nine matrix. We're going to end up with something like this and all the rest will have zeros. Maybe there's something special about the goal that if you're in . zero and that's your goal, maybe we put a large number here so that you don't move away from your goal once you've attained it. Okay, so that's what the reward matrix is going to look like. States down vertically and then actions along horizontally. So, depending on the state we're in, we can look at what actions are possible. Now, if I'm in room one, I can move to room zero and that may have the big reward. And so, what we might do with the reward matrix is have our goal as not just a one, but with some larger number like a 100.

Now what we're going to try and do is I want you to see if you can capture the geometry of this. So, we've got rooms zero, one, two, three, four, and five is the outside, but that will be like a large room if you like surrounding everything. Now, we can maybe simplify that geometry like this. So, here from one, you can go to five, which was the goal, the outside, or you can go to three. From two, you can only go to three. From three, you can go to one, or four, or two. So, what I want you to do is now to create this reward matrix of given a state, what action is possible? So, from state zero, where can I go? So, let me give you a clue here. So, here's state 0. I can only go to state four, so I'd have a one in state four. If it's possible, we're going to put a one there. So, here's state four. We'd have a one, but all the rest of this top row would be zeros except for that one in location four. So, I want you to complete this rewards matrix, and then we're going to go on from here and understand how we can do reinforcement learning and get

© MIT xPRO® 2024. All rights reserved.

our robot to make its way from anywhere that its dropped to the goal. So, that's your first task. Understand how to build up the geometry of what moves are possible. Okay, bye for now.

Video 5: Reinforcement Learning: The Quality Matrix and the Bellman Equation (10:36)

So, if you remember last time, we were doing reinforcement machine learning and we were trying to capture the geometry of a problem and we managed to get the layout here in a graph form and we decided that we'd flag whether an action was possible or not by a one. If it was possible, we'd have a one. If it's not possible, then zero. And what we mean by an action is moving from one room to another. So, for example, moving from room one to five is possible. Moving from room one to two is not possible.

So, we started out with state zero, so down here, and the only possible move is to four. So, we put a one corresponding to that action of moving from zero to four. Now, you needed to complete being in state one. What were the possible actions? Well, you could go to three or five. So, this was the table that you needed to complete. This is what I asked you to do. So, this is the solution and this is what we call our reward matrix, R. Now, we're going to add a wrinkle to this that we're going to add for our goal 100 units as a reward.

So, we're going to update our reward so that if you move from one to five, you get a reward of 100. If you move from four to five, you get 100. And we also, we add this one in that if we stay at five move from five to five, it's 100. Now, it'll be clear why we need this one in as we move on. Now what we're going to do is we're going to introduce another matrix, we're going to call it a quality of move matrix or Q-matrix for short. And this is going to specify, these are going to be our bread crumbs.

So, these numbers are going to be high in some areas and low in other areas. So, we know what direction we need to move in. So, let's take a look at this move. The quality of moving from one to five. So, it means we're dealing with this row here one to five. Now this is the formula we're going to apply is that the quality of being in state S and moving to A depends on the reward of moving from S to A. So, moving from one to five, we have a reward here of 100. So, this is going to be at least a 100.

Now, we choose gamma as being 0.5. I'm choosing 0.5 so that it makes the calculations easier.

© MIT xPRO® 2024. All rights reserved.

And now we look at where we move to the Q in the row of where we've moved to. So, we've moved to row five or state five. So, we look along this row and see if there are any Q values and we choose the largest. Now at the beginning, there aren't any here. So, we're just left with that, Q now will get updated to be 100, and this quantity is zero on the right. So, this becomes 100. Let's move on now.

So, the algorithm we're going to apply is we're going to choose a state at random, see what actions are possible, and calculate the Q value for one of those actions. So, let's we've seen Q for one to five. Let's now take a look at Q for one to three. So, here we are in one, and here's three. So, we know we've got a reward of one. Now it means that we've moved from one to three. Now, we need to look in row three to see if there's any other Q that contributes.

We're looking at this term here, this Q(s_next), s_next is three. And so we look at this row and there's no value there. So, we're just going to be left with that reward of one. So now, our Q gets updated in row one, and we've got a one here and 100 here. Let's go and see another action, three to one. So, here's three to one. We know we've got a reward of one there. Now let's see, we have to look in row one now to see if we've got any Q values. And we do. We've already got a one here and we've got 100, and we take the maximum one. So, we take the 100 multiplied by gamma, which is 0.5, so that's 50. So, and the one from our reward in our pocket, if you like, makes this 51. So, now we've got our Q value here of 51. Let's take another one, moving from two to three. So, here we are in two to three. So, we've got the reward of one. Now we need to look in row three to see what we've got there. Well, we've got a 51 there, so we take half of that because that's the maximum in that row. We take half of it, 25, I'm rounding down and then add that one.

So, we should get 26. You're beginning to see the pattern, feeling comfortable with this? Let's do another one, we'll do one to three again. Now this had a value of one already in there, but things have changed since then. So, now we need to take a look at the reward moving from one to three. So, that was a reward of one. Now, we look in row three and we see that the maximum Q in there is 51. So, we're going to get 25, when we multiply by the gamma, the half and adding one, this gets updated from one to 26. We'll just do a couple more, five to one. So, here we are. We're in row five, and we're going to move to row one. Now, five to one has a reward of one. So, this term, the R of the s, a is one. And now we need to look in row one to see what the maximum reward is, and it's 100. So, we're going to get 50 plus that one, 51. I'm just going to do

© MIT xPRO® 2024. All rights reserved.

one more. This is the last one, I promise you. Now, four to five. So, well, there's a reward of 100. So, this is already 100 of going from four to five. Now we need to look in row five to see the Q, maximum Q, and it's 51. So, we take half of that and add it to this. And this time I round it up. So, that gives 126. I just wanted to show you that we could, even though we start with 100 here, our maximum reward, this can increase because other numbers can contribute to it. So, we do this until this Q matrix is static and has converged and I ran it for 100 iterations and it converged to this. So, now our robot can find its way around.

Let's start the robot in row two. So, the robot starts in row two. It looks along here for actions that it can take, and it looks for the maximum quality and it sees a 52 here. So, it takes that. So, that means it's going to go to three. This action is move from two to state three. So, now we're in state three. We're here. Now, we're in this row and we look for the maximum action. In this case, we've got two that are the same. So, we could go to one or we could go to four and you could see that yes, those are good, those are equally possible and will get us to the goal. So, let's suppose we go to four, let's go to four first. So, we look along here and we see we've got two of 52s, but we've got the 200 here moving from four to five. That's the best quality move, and so we get to our goal. So, that's how Q-matrix, once we've got it, will help our robot find the goal. No matter where you put it, now it's got a recipe for getting to the goal. Let's see what it means in terms of code.

So, we're going to choose a random state. We're going to find a list of possible actions. Choose an action at random, scan that row of Qs and choose the maximum Q. Now, update the Q value using our formula, our Bellman's equation, so adding in whatever reward plus gamma times the maximum Q in that row. So, last time we ended up with our Q-matrix here, and we solved four to five. Now, I want you to update this Q-matrix as follows. I want you to find the solution of four to three and of zero to four. So, you need to find the Q values for both of these. Now make sure that you understand exactly what's going on because this is one of the most important things in reinforcement learning, understanding how we can create a map for the robot so it can find its way around. Okay, good luck. Bye for now.

Video 6: Implementing the Quality Matrix in Python (09:20)

So, let's look at the code for reinforcement learning and solving the problem of teaching the robot to find its way around these set of rooms. We saw that we could represent the rooms in a

© MIT xPRO® 2024. All rights reserved.

more compact graphical format here and you set up the reward matrix so that the robot would only move along the paths where there was a one. So, if we're in state one, we can only move to three or to five. So, state one, this is the move to five to our goal, and this would be the move to state three. And, we saw how to develop a Q-matrix that would have our state vertically and the actions that we could take along the horizontal.

So, this was a six-by-six matrix. And we saw how to develop the actual values that we've been needed in here to find our way around, and we were using Bellman's equation to do that. And we saw that the main algorithm to create the Q-matrix would run something like this, that we'd be iterating around a loop. So, we might go for 1000 iterations so that we could develop a stable Q-matrix one that had converged. We choose a random state, we find a list of possible actions, we choose one of the actions. And so that becomes our next row, if you like that we're going to go to. We scan that row for its any Q values in it, and then we update the Q according to this. We look at the reward matrix first, add any rewards in, and then we take gamma times the maximum reward for. In this case, if we were moving to row one or state one, it would be the maximum Q in that row. Okay, so let's go take a look at actual code. I'm going to show you the code in Visual Studio. I'm going to give it you in a Colab notebook, but I suggest you load it into Visual Studio, so you can run it in debug mode and debug it.

So, let's take a look at the code. So, here we are at the beginning. We need NumPy and we're setting up a problem with six dimensions. So, here's our rooms zero, one, two, three, four, five, and we define actions. And, in fact, we only need zero through five of those actions. And we set up a reward matrix. And this we do by hand. This represents the geometry. And remember, and to some extent we, it doesn't have to be 100 for the reward of the goal, you can choose whatever you like.

Some people use 999. So, now we choose a gamma, a discount factor, and we're going to choose a learning rate as well.

I've left it as one here, but it's usually less than one. And this is the function that does all the work, get optimal route. So, this we input a start location, and an end location, and we get the rewards matrix. We choose an ending state and here we set the ending state to have a reward of 100. Now, we're going to calculate the Q-matrix, it's six-by-six in our case. And now we go through the Q learning process. We have our for loop. We choose the current state, so we're

© MIT xPRO® 2024. All rights reserved.

going to choose a room to be in, and we choose it at random. We find the playable actions.

What that means is what other rooms can we move to if we're in this room. And we do that by checking whether the reward of moving from the current state to the next state is greater than zero. So, that's why we put ones in our matrix, our reward matrix. And even if when we put the 100, it's greater than zero, so that would be fine. So, we pick a random action, out of the playable actions, we choose one. Now, this does all the work. This is called temporal difference, but it's basically the Bellman equation. Here you see the gamma, we have here that we get the next state.

So, that's now we're picking out a row of where we're going to. And now, we look along that row for the maximum, we choose the maximum argument. So, we're choosing the maximum Q value. So, we get that maximum Q value, we subtract off whatever current, the Q value of the current state next state is. So. if we've got a value already, we're taking that away from whatever we calculate, but it gets added in here in this update, we're going to take alpha. So, alpha is another learning rate. We've got the gamma one and we've got alpha now is a learning rate, we take some fraction of the temporal difference. And it's just because as we loop through this Q-matrix, we'll get new values added. So, the same calculation that we've already done might actually give a different value later. We saw that our maximum reward was going up from 100, and eventually it went up to 200. So, this does all the work, and then we keep looping around to that for however many iterations. I suggest you start with five so you can actually step through it.

So, here we'd put five, that's going to control our iterations. We can put a break point. We can run in debug mode. So here, so we run in debug mode and you see we get here and now we can step through. So, the current state, we're choosing five. It's actually we're at the goal where we want to be, but that's okay. And then we loop over the playable actions. So, the next state we choose here is one. Now, we find out what our reward is. So, here TD is one, it's allowable, we've got some reward. There are no queue states yet, they're all zero. And now, we're going to update five to one with the value of alpha, which is one times one.

Here now, we'll see things printed out, yes. Now, we're going, and we should be at the end here. We've done our loops so we can end it. So, that's how you work through in debug. Why don't you run this? And as an exercise, I want you to change the alpha to 0.9. So, here we'll have an alpha

© MIT xPRO® 2024. All rights reserved.

of 0.9, and change gamma to 0.75. And I want you to run it for 1000 steps and print out the matrix, the Q-matrix at the end, the stable Q-matrix. So, that's your exercise. I'll give you the link to where the code is. As I say, I would run this in Visual Studio if you want, though you're perfectly allowed to run it in Colab. Okay, bye for now.

Video 7: Reinforcement Learning Exercise (00:38)

So, in this exercise, I want you to retrain the code on this problem. So, I've altered this slightly from the one we showed before, but you need to make sure that the robot can start anywhere, but find its way to the goal. So, I want you to develop the Q-matrix and develop the reward matrix so that it can solve this problem. Okay, good luck. Bye for now.

Video 8: Reinforcement Learning Solution (02:39)

So, the goal was to train the robot so that it could find the goal, which is room zero here. And, as a sample, we'll start it at room eight. So, the first thing we need to do is to get the geometry into the reward matrix. So, this is the reward matrix that I found. So, on row zero is connected to one and to three, and one is connected to zero, two, and four, etc. So, we need to get the geometry correct and once we've got that, we can put it into our code. I'll show you the code that I have.

We start out with NumPy, and we need to get our location sorted out. This is just syntactic sugar really. So, here's the new room layout. And now the code, I'm running with gamma 0.75 and alpha 0.9. This code stays exactly the same. I put 100 in there for the reward at the ending state, the goal, but everything else is the same. And, the only thing we need to alter is the start and the end point. And, I'm starting at eight and going to room zero. And, here's my Q-matrix being printed out. I'll run this just in case you ran it with one and 0.5 for these. So, gamma 0.5 and alpha as 1.0. Let's just run it and we should get a very quick solution here. So, this is with gamma as 0.5 and alpha as 1.0. But we get basically the same root, so the robot does just as well with either of those choices. Okay, I hope you enjoyed learning about reinforcement learning; it's a very powerful technique indeed. Okay, bye for now.

Video 9: Applying Reinforcement Learning: The Frozen Lake Game (01:24)

Now that you've done some exercises in reinforcement learning and you understand how the Q-Matrix works, take a look at this quick example here called Frozen Lake. It's a fun game that you can find on the web. In this game, the person needs to navigate across the lake without falling in

© MIT xPRO® 2024. All rights reserved.

these holes. So, part of the lake is frozen and part of the lake has melted, and we need to reach this goal.

You'll be familiar with the Q-learning matrix. So, in this case, there are 16 different states and each state has four actions. And so, we've got a 16 by 4 Q-learning table. There's some randomness injected into the game, so you won't always win. So, run the game, change some of the hyper parameters such as the gamma and the alpha and see how good your robot is. Here, I reached the goal a few times, but I fell into the hole just as many. Okay. Obviously, I need to have it learn better. So, enjoy Q-learning with the Frozen Lake. Bye for now.

Video 10: Deep Dream Exercise (03:52)

So, I thought it'd be a fun exercise to take a look at a deep neural network and understand a little bit about the layers and what those weights in the neural network represent. So, there's a famous implementation called DeepDream. A gentleman called Alexander Mordvintsev developed and it's implemented here in Google Colabs. And basically what we need to do is we need to modify it so that we can take an image such as this where I feed in myself and then we have it operated on by the deep neural network, and this is how it comes out.

So, I want you to load up one of your images. So, follow the instructions because we need to modify the code, but it should give you an interesting view of yourself as a deep neural net sees you. So, the first thing we need to do is to upload an image of ourselves. So to do that, go here and click on this. And now, we can upload an image here just by drag and drop. So, I'm going to take my image and just drag it onto here. So, once we've loaded the image, we need to copy its path, then scroll down till you find 'URL =' on this line here.

And, we're going to substitute our path that we've just copied for that URL. So, it'll be /content/whatever your image is. Now down below here, we're going to comment out image path. And instead of the image path here, we're just going to use the URL that we've just copied in. So, it's going to pick up our image. So, now you can execute the code and everything should run. Let's go through and just check, so those are all loaded, load this, okay, and it displays the image. So, this is my uploaded image.

Now we need to go through, and now, it's going to actually operate. Here we go, DeepDream model, now the main loop, and calculate the image. So, this is the one I've got. We can alter the

© MIT xPRO® 2024. All rights reserved.

step size here on the number of steps, but basically that's DeepDream and how you get very strange images. So, you can play around with that, but I think it gives you some insight into the way that deep learning is operating. Okay, hope you have fun with this. Don't forget to put this up into your portfolio. I think these images are quite interesting. Okay, bye for now.

© MIT xPRO® 2024. All rights reserved.